

Tecnatura Superior en Desarrollo de Software

Anexo 05b — Dictionary<TKey,TValue> y HashSet<T> en C#

Asignatura: Programación II | Instituto de Educación Superior N.º 9007 Dr. Salvador Calafat |
Ciclo Lectivo 2026

¿Qué es?

`Dictionary<TKey,TValue>` es una colección de pares clave-valor que garantiza que cada clave sea única. Permite acceder a un valor en tiempo $O(1)$ promedio a partir de su clave, sin necesidad de recorrer la colección. Implementa `IDictionary<TKey,TValue>` y utiliza internamente una tabla hash para organizar los datos.

`HashSet<T>` es una colección de elementos únicos sin orden garantizado. Implementa `ISet<T>` y también usa una tabla hash internamente. A diferencia del diccionario, no almacena valores asociados: solo el elemento en sí. Su principal fortaleza es verificar pertenencia y realizar operaciones de teoría de conjuntos en tiempo constante o lineal.

Analogía

Imagine un casillero de correo en un edificio. Cada casillero tiene un número de apartamento (la **clave**) y contiene las cartas del residente (el **valor**). Para buscar las cartas del apartamento 4B, no hace falta abrir todos los casilleros: se va directo al número. Eso es un `Dictionary<TKey,TValue>`: acceso directo sin búsqueda lineal.

Un `HashSet<T>` es como una lista de invitados a un evento. Solo importa si alguien *está* o *no está* en la lista, no hay información adicional por persona. Verificar si "Juan" está invitado es instantáneo, y agregar a alguien que ya figura no genera duplicado.

Características

- **Acceso $O(1)$ promedio:** Tanto la búsqueda como la inserción y eliminación son tiempo constante en el caso promedio, gracias a la tabla hash interna.

- **Claves únicas:** En `Dictionary`, no pueden existir dos entradas con la misma clave. En `HashSet`, no pueden existir dos elementos iguales.
- **Sin orden garantizado:** Ninguna de las dos estructuras garantiza el orden de iteración. Si se necesita orden, existen `SortedDictionary` y `SortedSet`.
- **Operaciones de conjunto:** `HashSet<T>` expone `UnionWith`, `IntersectWith`, `ExceptWith` y `IsSubsetOf`, que operan directamente sobre la estructura sin crear copias intermedias.
- **Dependencia de GetHashCode y Equals:** El comportamiento correcto depende de que el tipo de clave (o elemento) implemente correctamente estos dos métodos.

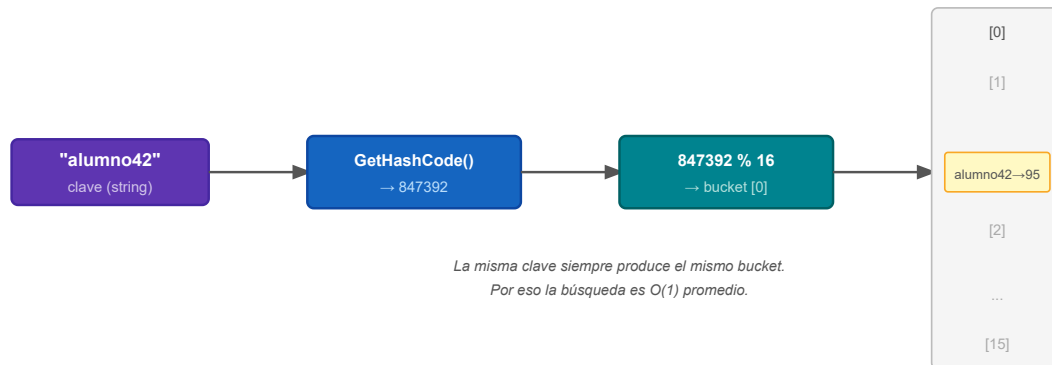
¿Por qué se usan?

Cuando se necesita buscar un elemento por un identificador conocido (nombre, código, ID), una lista requiere recorrer todos los elementos hasta encontrarlo: $O(n)$. Un diccionario calcula directamente en qué posición de memoria buscar: $O(1)$ promedio. Para colecciones grandes, esta diferencia es decisiva.

`HashSet<T>` resuelve el problema de eliminar duplicados y verificar pertenencia de manera eficiente. Convertir una lista con posibles repetidos en un `HashSet` elimina duplicados automáticamente. Verificar si un elemento pertenece a un conjunto es $O(1)$ frente al $O(n)$ de `List.Contains()`.

Cómo funciona internamente: tabla hash

Ambas estructuras usan el mismo mecanismo interno: una **tabla hash**. Cuando se inserta una clave (o un elemento en el caso de `HashSet`), el runtime llama a `GetHashCode()` para calcular un número entero. Ese número se convierte en un índice de un arreglo interno (llamado bucket). El elemento se almacena en ese bucket.



Si dos claves distintas producen el mismo bucket (colisión), la estructura las encadena dentro del mismo bucket. En el peor caso teórico, todas las claves colisionan y la búsqueda se degrada a $O(n)$. En la práctica, con tipos bien implementados, las colisiones son raras y el rendimiento se mantiene constante.



Atención: Si se usa un tipo propio como clave de un diccionario, es obligatorio sobrescribir `GetHashCode()` y `Equals()` de forma consistente. Dos objetos que son *iguales* según `Equals` deben producir el mismo hash. Si no, el diccionario no encontrará claves que deberían existir.

Marco teórico

Dictionary<TKey,TValue> — operaciones esenciales

Miembro	O	Descripción
<code>dict[key] = value</code>	$O(1)^*$	Inserta o actualiza el valor asociado a la clave.
<code>dict[key]</code>	$O(1)^*$	Devuelve el valor. Lanza <code>KeyNotFoundException</code> si no existe.
<code>TryGetValue(key, out val)</code>	$O(1)^*$	Devuelve <code>true</code> y asigna el valor si la clave existe. No lanza excepción.
<code>ContainsKey(key)</code>	$O(1)^*$	Verifica si la clave existe sin acceder al valor.
<code>Remove(key)</code>	$O(1)^*$	Elimina la entrada con esa clave.
<code>Keys / Values</code>	$O(1)$	Devuelve colecciones de solo lectura de claves o valores.

Miembro	O	Descripción
Count	O(1)	Número de pares almacenados.

* O(1) promedio. En caso de colisiones extremas puede degradar a O(n).

```

1 // Construcción e inserción
2 var scores = new Dictionary<string, int>();
3 scores["Ana"] = 92;
4 scores["Luis"] = 85;
5 scores["Marta"] = 78;
6
7 // Acceso seguro con TryGetValue (patrón recomendado)
8 if (scores.TryGetValue("Ana", out int nota))
9     Console.WriteLine($"Ana obtuvo {nota}");
10
11 // Iteración sobre pares
12 foreach (var kvp in scores)
13     Console.WriteLine($"{kvp.Key}: {kvp.Value}");

```

HashSet<T> — operaciones esenciales

Miembro	O	Descripción
Add(item)	O(1)*	Agrega el elemento. Devuelve <code>false</code> si ya existía (sin excepción).
Contains(item)	O(1)*	Verifica pertenencia. Mucho más rápido que <code>List.Contains()</code> para sets grandes.
Remove(item)	O(1)*	Elimina el elemento si existe.
UnionWith(other)	O(n)	Agrega todos los elementos de <code>other</code> que no estaban. Modifica el set actual.
IntersectWith(other)	O(n)	Conserva solo los elementos que también están en <code>other</code> .
ExceptWith(other)	O(n)	Elimina los elementos que están en <code>other</code> .
IsSubsetOf(other)	O(n)	Devuelve <code>true</code> si todos los elementos del set están en <code>other</code> .

```

1 // Eliminar duplicados de una lista
2 var tags = new List<string> { "c#", "dotnet", "c#", "oop", "dotnet" };
3 var unique = new HashSet<string>(tags); // { "c#", "dotnet", "oop" }
4
5 // Operaciones de conjunto
6 var grupoA = new HashSet<string> { "Ana", "Luis", "Marta" };

```

```

7  var grupoB = new HashSet<string> { "Luis", "Pedro", "Marta" };
8
9  grupoA.IntersectWith(grupoB); // grupoA = { "Luis", "Marta" }
10 // Alumnos que cursaron en ambos grupos

```

Dictionary vs List: cuándo elegir cada uno

Criterio	List<T>	Dictionary<TKey,TValue>
Buscar por posición	✓ O(1) por índice	✗ No aplica
Buscar por valor/clave	O(n) — recorre toda la lista	✓ O(1) promedio
Orden de inserción	✓ Garantizado	✗ No garantizado
Duplicados	✓ Permite	✗ Claves únicas
Uso de memoria	Menor overhead	Mayor overhead por tabla hash

Errores comunes

ERROR COMÚN

Acceder con `dict[key]` sin verificar si la clave existe.

```
1 var nombre = dict["carlos"]; // KeyNotFoundException si no existe
```

Si la clave no existe, se lanza `KeyNotFoundException` en tiempo de ejecución, interrumpiendo el programa.

BUENA PRÁCTICA

Usar siempre `TryGetValue` para acceso defensivo.

```

1  if (dict.TryGetValue("carlos", out var nombre))
2      Console.WriteLine(nombre);
3  else
4      Console.WriteLine("No encontrado");

```

Una sola operación: verifica existencia y recupera el valor al mismo tiempo, sin riesgo de excepción.

Qué NO hacer

QUE NO HACER

Usar `List.Contains()` para verificar pertenencia cuando la colección es grande y se consulta frecuentemente.

```
1 var legajos = new List<int>(); // 10.000 elementos
2 if (legajos.Contains(5432)) { ... } // O(n) cada vez
```

Con 10.000 elementos y 1.000 consultas, esto ejecuta hasta 10 millones de comparaciones. Es un error de diseño frecuente en aplicaciones con volumen de datos.

BUENA PRÁCTICA

Usar `HashSet<T>` cuando la operación principal es verificar pertenencia.

```
1 var legajos = new HashSet<int>(); // 10.000 elementos
2 if (legajos.Contains(5432)) { ... } // O(1) promedio
```

Con la misma cantidad de datos y consultas, el `HashSet` realiza exactamente 1.000 operaciones en lugar de hasta 10 millones.